

LeetCode 1331: Rank Transform of an Array

Easy

Solution Analysis & Implementation Guide

1. Problem Overview

Objective: Given an array of integers `arr`, replace each element with its rank.

Ranking Rules:

- Ranks should start from **1**.
- The larger the element, the larger the rank. If two elements are equal, their rank must be the same.
- Ranks should be as small as possible (consecutive integers without gaps).

2. C++ Source Code

```
class Solution {
public:
    vector<int> arrayRankTransform(vector<int>& arr) {
        int n = arr.size();
        vector<int> ans(n);

        vector<pair<int, int>> helper;
        for(int i = 0; i < n; i++) {
            helper.push_back({arr[i], i});
        }

        sort(helper.begin(), helper.end());
        int curr = 1;

        for(int i = 0; i < n; i++) {
            int val = helper[i].first;
            int idx = helper[i].second;

            if(i == 0) {
                ans[idx] = 1;
                continue;
            }

            if(i > 0 && val == helper[i-1].first) {
                ans[idx] = curr;
                continue;
            }

            curr++;
            ans[idx] = curr;
        }

        return ans;
    }
};
```

3. Algorithm Notes & Breakdown

To assign ranks while preserving the original order of the array, we must keep track of each element's initial index before sorting. This implementation does exactly that using an auxiliary vector of pairs.

Step-by-Step Execution

1. **Pairing with Indices:** We initialize a helper vector that stores pairs of (value, original_index). This allows us to map the elements back to their correct positions in the ans array after sorting.
2. **Sorting:** We sort the helper vector in ascending order based on the values. By default, C++ sorts pairs by their first element, which aligns perfectly with our need to order the values.
3. **Rank Assignment:** We iterate through the sorted helper array. We maintain a curr variable starting at 1 to represent the current rank.
 - For the first element ($i = 0$), we assign the rank 1 to its original index in the ans array.
 - For subsequent elements, we check if the current value matches the previous value. If it's a duplicate, it receives the exact same curr rank.
 - If it's strictly greater, we increment curr and assign the new rank.

4. Complexity Analysis

Time Complexity: $O(N \log N)$

Creating the helper array takes $O(N)$. Sorting the array of pairs is the bottleneck, taking $O(N \log N)$. The final iteration to assign ranks takes $O(N)$. Thus, the overall time complexity is dominated by the sorting step.

Space Complexity: $O(N)$

The space complexity is $O(N)$ because we allocate memory for the ans array of size N and the helper vector of pairs, which also takes $O(N)$ space.